Lab 09

Multiplexer and Demultiplexer

Name:	Student ID:

9.1 Objective

After performing this lab, students should be able to:

- Design and test Verilog modules for Multiplexer and Demultiplexer
- Identify the difference between Gate Level and Dataflow modeling
- Demonstrate data multiplexing through digital logic

9.2 Introduction

In this lab, you're going to build a system that is capable of driving 4 common anode seven segment displays with multiplexed data lines, to display different numbers on the each seven-segment LED display, we have to control the cathodes (YA-YG) of the four seven-segment LEDs separately and by activating the four seven-segment LEDs at different times. For example, when we activate Display 1 by driving the common anode terminal other three Displays (i.e., Display 2,3,4) will be deactivated using Enable lines, see Figure 9.1.

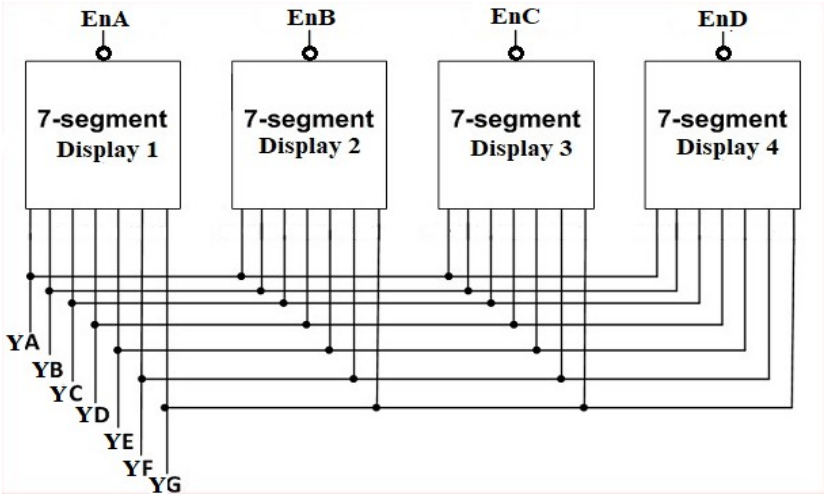


Figure 9.1

The overall system accepts 4-bit data in four data buses A, B, C and D for each seven-segment display, 2-bit data for the select line S to control the 4-bit data, which is passed to the binary to hexadecimal decoder1, the final output (7-bit data) of which goes to the seven-segment display and the seven segment LEDs are controlled by Enable bit to enable or disable the display of selected seven segment display.

Use Binary to hexadecimal module already created in Lab 6.

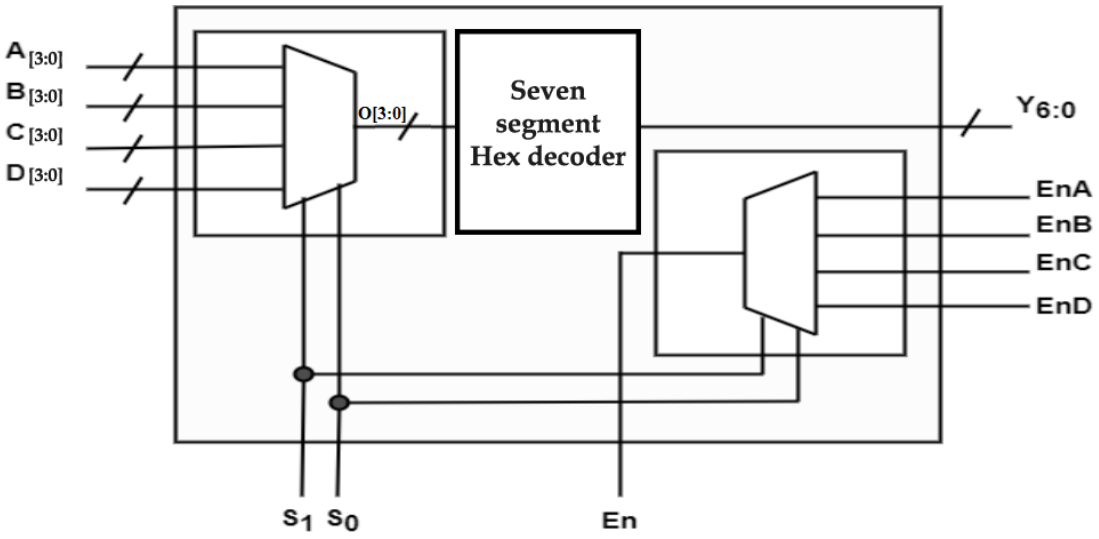


Figure 9.2 Top Level Module

In the next two sections, you'll implement the multiplexer and demultiplexer circuit and then connect them together to work as a single module.

9.3 Multiplexer

A multiplexer (MUX) is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line. The selection of a particular input line is controlled by a set of selection lines. Normally, there are 2n input lines and n selection lines whose bit combinations determine which input is selected and its logic state (i.e., 0 or 1) is transferred to output. Figure 9.3 shows the symbol of general multiplexer

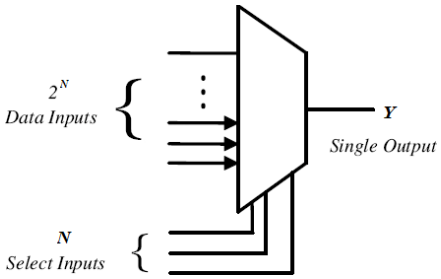
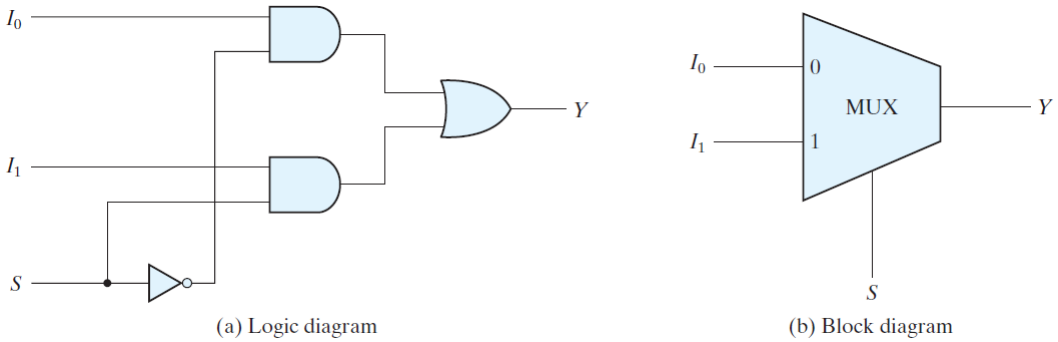


Figure 9.3

A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination, as shown in Figure 9.4 The circuit has two data input lines, one output line, and one selection line S.

When *S*=0, the upper AND gate is enabled and *I*0 has a path to the output. When *S*=1, the lower AND gate is enabled and *I*1 has a path to the output.



(a) Logic diagram (b) Block diagram

Figure 9.4

Implementations of 2:1 MUX

The simplest multiplexer is the **2:1 MUX** which simply selects its output from just two possible inputs. Its selection lines are therefore made of a single bit.

Table 9.1

S	I_0	I_1	Y
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

Note: 'x' in input columns as in Table 9.1 means, the variable can be 0 or 1. Do not confuse those as "don't care" conditions in k-maps which are represented as "X" in output column.
The logic function for a 2:1 MUX can be derived using the min-terms from Table 9.1

$$Y(S,Io,I1) = (Io\bar{S})+(I1S)$$

Gate Level Vs Dataflow modeling

Gate Level Modeling is the low level or 'Gate Level' abstraction. At gate level, the circuit is described in terms of gates and their instantiation. Hardware designer in this level requires basic knowledge of digital logic design. Verilog supports basic logic gates as predefined "primitives". All logic circuits can be designed by using basic gates. For example

```
not M1(y, a);
and M2(y, a, b);
or M3(y, a, b);
```

Modeling in Gate level is very difficult for a complex circuit design. Data flow modeling provides a powerful way to implement a design. Verilog allows a circuit to be designed in terms of the dataflow between registers and process data or expression rather than instantiation of individual gates. This approach allows a designer to concentrate on optimizing the design in terms of data flow.

For maximum flexibility in the design process, designers typically use a Verilog description style that combine the concepts of Gate-level, and Data- flow design.

Conditional Operator

Conditional operators are frequently used in dataflow modeling to model conditional assignments. The conditional expression acts as a switching control.
The conditional operator (? :) takes three operands.

Usage: *condition_expr ? true_expr : false_expr;*

The condition expression (condition_expr) is first evaluated. If the result is true (logical 1), then the true_expr is evaluated. If the result is false (logical 0), then the false_expr is evaluated

If the result is x (ambiguous), then both true_expr and false_expr are evaluated and their results are compared, bit by bit, to return for each bit position an x if the bits are different and the value of the bits if they are the same.

The action of a conditional operator is similar to a multiplexer. Alternately, it can be compared to the if-else expression.

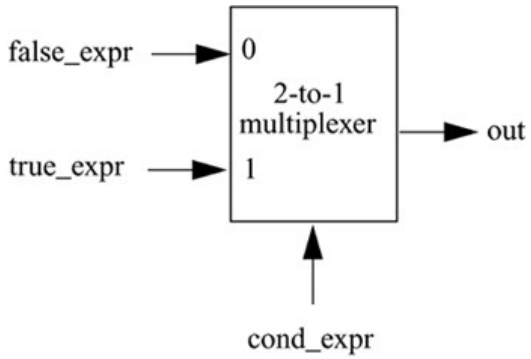


Figure 9.5

for example, 2:1 Mux in Figure 9.4 can be implemented as follows

```
//model functionality of a 2-to-1 mux
assign Y = S ? I1 : I0;
```

Task a)

In the previous lab (Lab 6) you have formulated a truth table for an active low decoder for (common anode) seven segment display, which will be used in the later part of this Lab. In this task you will develop a 4:1 MUX using conditional operator that will receive 3-bit binary data representing 4 digits of your registration ID and the output will be selected through the selection lines for each digit as shown in Figure 9.6.

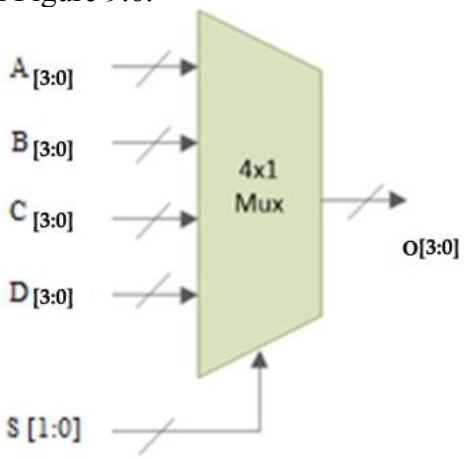


Figure 9.6

Conditional operations can be nested in two ways. Each true_expr or false_expr can itself be a conditional operation. For example

```
assign O = S[1] ? (S[0] ? D :C) : (S[0] ?B :A);
                                                Or
assign O = (S==2'b00) ? A : (S==2'b01) ? B : (S==2'b10) ? C : D;
```

Complete the truth table provided in Table 9.2. As an example, when the Select line input is S1:0=00, output will be the binary number stored in A3:0

Table 9.2

S_1	S_0	$A_{3:0}$	$B_{3:0}$	$C_{3:0}$	$D_{3:0}$	$O_{3:0}$
0	0		*x*	*x*	*x*	
0	1					
1	0					
1	1					

Write a Verilog module for 4:1 MUX for Table 9.2 using conditional operator in Vivado and design a test bench to test the designed module for all possible values of S_1S_0 as per Table 9.2. Name your module as **mux_4_1.**

9.4 Demultiplexer

The function of Demultiplexer (DeMUX) is opposite to multiplexer function. It takes information from one line and distributes it to a given number of output lines. For this reason, the demultiplexer is also known as a data distributor.

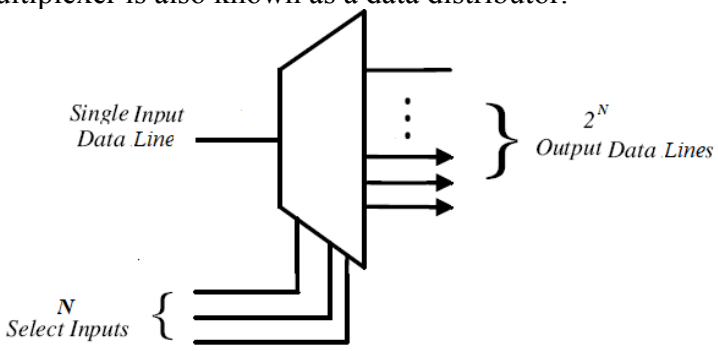


Figure 9.7

Implementations of 1:2 DeMUX

A 1:2 demultiplexer uses 1 select line (*S*) to determine which one of the 2 outputs (*Y*0−*Y*1) is routed from the input (*D*). Its characteristics can be described in the following simplified truth table.

Table 9.3

S	D	Y_1	Y_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

The logic function for a 1:2 MUX can be derived using the min-terms from Table 9.3

$$Y0 = \bar{S}D$$
$$Y1 = SD$$

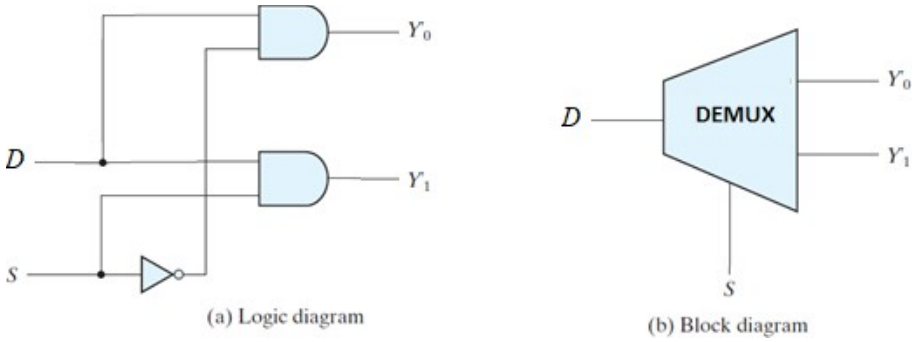


Figure 9.8

Task b)

So far, we have transmitted the data of our desired digit to the multiplexed data line O[3:0] of binary to hexadecimal converter by selecting it through the select lines. In this task we will use 1:4 DeMux to enable one of 4 seven segment displays to show the result on seven-segment of our choice.

As discussed earlier each of the seven-segment display is Common Anode (CA), which require Logic 0 at the common terminal to work, but in figure 9.1 we can see an 'inversion bubble' at the CA terminal of each seven segment. Which mean we have to design a DeMux that work on active low logic.

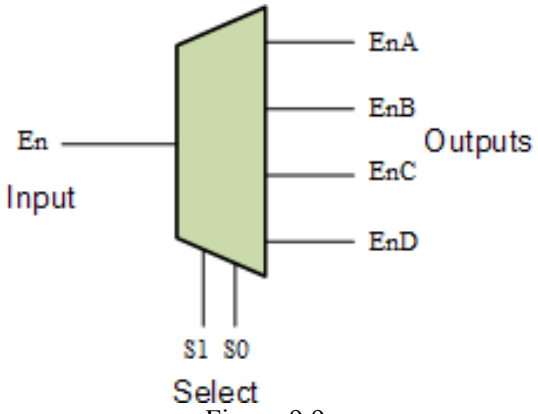


Figure 9.9

Complete the truth table provided in Table 9.4. As an example, when the Select line input is S1:0=00, output EnA will be 0 and the rest of the outputs will remain high.

Table 9.4

S_1	S_0	*En*	*EnA*	*EnB*	*EnC*	*EnD*
0	0	0	0	1	1	1
0	1	0				
1	0	0				
1	1	0				
x	*x*	1	1	1	1	1

You can use conditional operator for each output equation such that assign output_value1 = select_signal ? enable : 1'b1

Write a Verilog module for 1:4 DeMUX for Table 9.4 using conditional operator in Xilinx Vivado and design a test bench to thoroughly test the designed module. Name your module as demux_1_4

Task c)

Write a Verilog module topLevelModule that combines MUX **(mux_4_1)** and DeMUX (demux_1_4) modules created in this lab and binary-hexadecimal decoder module already created in Lab 6, according to the block diagram of Figure 9.2 Top Level Module. This module's inputs includes four 4-bit data lines (i.e., A[3:0], B[3:0], C[3:0], D[3:0]), 2-bit select line (i.e. S[1:0]) to select from the data for binary-hexadecimal decoder, and a 1-bit active low enable signal (i.e. En).

The modules output includes one 7-bit data line for seven segments (i.e., Y [6:0]) and four 1-bit Enable signals (i.e., EnA, EnB, EnC, EnD).

9.5 Exercise (Post Lab)

a) Add code of the topLevelModule

b) Create testbench for your topLevelModule and test each line by holding binary coded decimal number of your student registration ID.

c) Implement your design on BASYS-3 board to demonstrate data multiplexing. Note that, on Basys-3 board there are 16 input switches and the total number of input bits in this design are nineteen (19), in order to test your code on board hardcode input D[3:0] in your Verilog topLevelModule.

Assessment Rubric
Lab 09
Multiplexer and Demultiplexer

Name:	Student ID:

Marks Distribution:

		LR2 Code	LR5 Results	LR9 Report
In - Lab	Task a	10	10	20
	Task b	10	10	
	Task c	10	5	
Exercise		10	5	
Total Marks: 100		/40	/30	/20
CLO Mapping		CLO2	CLO2	CLO4

Affective Domain Rubric		Points	CLO Mapped
AR 7	Report Submission and Viva	/10	CLO 4

CLO	Total Points	Points Obtained
2	70	
4	30	
Total	**100**	

For description of different levels of the mapped rubrics, please refer to the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

Lab Evaluation Rubrics

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR2	**Program/Code/ Simulation Model/ Network Model**	Program/code/ simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/ simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/ simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/ simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	**Results & Plots**	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	**Viva**	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR9	**Report**	No summary provided. The number/amount of tasks completed below the level of satisfaction	Couldn't provide good summary of in-lab tasks. Some major tasks were	Good summary of In-lab tasks. All major tasks completed except few minor	Outstanding Summary of In-Lab tasks. All task completed and

		and/or submitted late	completed but not explained well. Submission on time. Some major plots and figures provided	ones. The work is supported by some decent explanations, Submission on time, All necessary plots, and figures provided	explained well, submitted on time, good presentation of plots and figure with proper label, titles and captions